

Sapphire Coding and Usage Guide

Language syntax, standard-library namespaces, CLI usage, and verified boundaries.

This manual describes the current local implementation, tested behavior, and honest boundaries of Sapphire. It is intentionally detailed so the document remains readable and useful beyond a short summary.

Overview

Sapphire is a lightweight interpreted language designed around local scripting, automation, and experimentation. It is not marketed as a finished distributed AI platform or a certified production runtime, and the repository is intentionally transparent about what has been validated and what remains future work.

The language is designed to be readable and approachable for Windows users, while still exposing enough structure for process control, file access, scheduling, and AI-assisted automation. The interpreter is intentionally compact, so the practical value comes from combining a simple syntax with controlled system access and optional integrations.

Core syntax

Variables are declared with `let` and `const`, with a syntax that supports optional type annotations when the author wants them. Functions can accept arguments, return values, and use closures or nested definitions; control flow includes `if`, `else`, loops, and exceptions. Arrays, maps, structs, and lambdas are all part of the language model used by the current interpreter.

The language also supports pipeline-style expressions and indexing, plus special handling for process literals using backticks or shell-style `$` access. These features are intended to make scripts feel natural for automation, especially in a local Windows environment where shell commands and file operations are common.

Stdlib namespaces

The runtime exposes several namespaces that are expected to be familiar to a developer writing local tooling: `os`, `fs`, `http`, `ai`, `data`, `scheduler`, `ml`, `gui`, and `agent`. Each namespace is independent and not all services are automatically available; the repo distinguishes between language features and optional runtime dependencies.

For example, `os` gives system and process information, `fs` handles files, `http` can be used when `requests` is installed, and `scheduler` manages local tasks. The `ai` and `ml` modules depend on configured backends or installed packages, and the `agent` namespace adds memory and planning features that are intentionally bounded by permissions and budgets.

Command-line usage

The CLI supports running `.sp` scripts, REPL-style input, and interpreter inspection commands. Typical usage includes `sapphire run` for script execution, `sapphire info` for environment details, and the bundled compiler or studio tools for inspection and editing. The project does not treat the compiler as a native code generator; it is a runtime and inspection tool that is useful for development workflows.

Developers should expect that the behavior of the runtime depends on the installed Python environment, the optional packages, and the local hardware. The project is transparent that capabilities can vary by machine and that notebook-style or cloud-style assumptions are not built into the core interpreter.

Example workflow

A practical script usually starts by reading system state, then performing a controlled operation, then storing or reporting the result. For example, a script can gather CPU telemetry, decide whether a condition is true, and then send a notification or write a log file. This style matches Sapphire's emphasis on local automation tasks rather than remote orchestration or massive distributed execution.

The example patterns in the project are intentionally conservative: script execution is local, tool access is tracked, and outputs are validated by direct logic rather than by assuming that a cloud or model call is always trustworthy. That approach keeps the language approachable while retaining a clear boundary between tested and untested behavior.

Version 1.0.7. This document is intentionally longer than a brief summary to provide context, operational detail, and practical caveats for local use. Values and capabilities depend on the machine, software environment, and installed dependencies.